

Bases de Datos 2 – TP4

Redis

Para este trabajo, se utilizo Redis a través de contenedores Docker, donde se utilizo la versión más reciente al momento (19/06/2025), la cual es la 8.0.2.

Primero, se bajó la imagen de Redis (docker pull redis), luego se instanció el contenedor a través de la imagen descargada, corriendo desde el puerto 6379 (docker run --name redis-server -p 6379:6379 -d redis). Con esto ya se tiene el contenedor funcionando, y luego ejecutamos “docker exec -it redis-server redis-cli” para ejecutar la interfaz interactiva de Redis a través del contenedor y poder empezar a realizar el trabajo.

Parte 1: Introducción

1. ¿Qué tipo de bd es Redis?

Redis una Base de Datos NoSQL de tipo clave-valor, donde todo datos que se almacena es una par clave-valor, donde el valor es accesible u obtenible a través de su clave.

2. ¿Dónde almacena los datos Redis?

Redis almacena los datos en memoria principal, sirviendo como una Base de Datos que funciona como una caché.

3. ¿Qué tipos de datos posee Redis?

Redis soporta varios tipos de datos, lo que le permite manejar diferentes estructuras y casos de uso complejos. Los tipos principales son:

- **String:** Secuencias de bytes simples que pueden contener texto, números, objetos serializados, imágenes, entre otros. Es el tipo más básico y usado.
- **List:** Listas ordenadas de strings, similares a arrays o listas enlazadas. Permite operaciones como push, pop y rango.
- **Set:** Conjuntos no ordenados de strings, donde cada elemento es único. Permite operaciones de unión, intersección y diferencia.
- **Sorted Set (ZSet):** Conjuntos ordenados donde cada elemento tiene un "score" (puntaje) que define su orden. Muy útil para rankings, colas con prioridad y ordenamientos.
- **Hash:** Colección de pares campo-valor, similar a un diccionario o mapa. Ideal para representar objetos con atributos.
- **Stream:** Estructura de datos para almacenar secuencias ordenadas de mensajes o eventos, útil para mensajería y procesamiento en tiempo real.
- **Bitmap:** Permite manipulación a nivel de bits dentro de un string, útil para estadísticas, flags y contadores eficientes.

- **HyperLogLog:** Estructura probabilística para contar elementos únicos (cardinalidad) de forma aproximada y eficiente en memoria.
- **Geospatial:** Soporta almacenamiento y consultas de datos geográficos, como coordenadas, para cálculos de proximidad y distancias.

4. Enuncie las características de Redis

- **Buen rendimiento a comparación de otras BD:** como mencionamos anteriormente, Redis cuenta con buen rendimiento ya que almacena los datos en memoria principal, lo que elimina la latencia asociada con el acceso a disco.
- **Estructuras de datos ricas:** además de strings, Redis soporta listas, conjuntos (sets), conjuntos ordenados, hashes y HyperLogLogs, permitiendo almacenar una amplia variedad de tipos de datos.
- **Replicación:** Redis permite la replicación maestro-esclavo, lo que aumenta la disponibilidad y permite la creación de copias de seguridad.
- **Persistencia:** aunque sea una base de datos en memoria, Redis ofrece opciones de persistencia para guardar los datos en disco y recuperarlos en caso de fallo.
- **Compatibilidad con múltiples lenguajes:** Redis es compatible con una gran cantidad de lenguajes de programación, incluyendo Java, Python, PHP, C#, JavaScript, entre otros.
- **Transacciones atómicas:** Redis proporciona comandos para ejecutar operaciones atómicas, lo que asegura la integridad de los datos en ciertas situaciones.
- **Escalabilidad:** Redis puede escalar horizontalmente mediante particionamiento o clústeres, permitiendo manejar grandes volúmenes de datos.
- **Publicación/Suscripción:** Redis soporta patrones de publicación/suscripción, lo que lo hace útil para sistemas de mensajería.
- **Claves con expiración automática:** Permite establecer un tiempo de vida (TTL) para las claves, útil para el almacenamiento en caché y sesiones.

5. Comparar Redis con los RDBMS

- A comparación de un RDBMS, no es necesario establecer un modelo ya que al almacenarse en memoria y al ser una base de datos clave-valor, el modelo resulta innecesario ya que se busca un dato a memoria mediante su clave.
- Es mucho más rápido que un RDBMS, ya que al almacenarse en memoria principal no es tan costoso la búsqueda de datos, además que al no existir un modelo o relaciones, no existen los joins.

- Es mucho más flexible que un RDMBS, ya que como se menciono antes, al almacenar los datos en formato de string y al no tener un modelo al que respetar, permite almacenar diversos tipos de datos.
- Facilita la extensibilidad o modificación de la aplicación, ya que al no tener un modelo que respetar, es más fácil de modificar y extender.

6. ¿Redis tiene transacciones?

Redis cuenta con transacciones. Las mismas, permiten la ejecución de un grupo de comandos en un solo paso. Los comandos que permite ejecutar dentro de una transacción son: MULTI, EXEC, DISCARD, y WATCH.

Además, las transacciones en Redis ofrecen 2 garantías:

- Los comandos de una transacción son serializados y ejecutados secuencialmente. Una request enviada por otro cliente nunca será atendida mientras se esté ejecutando una transacción, lo cual garantiza que los comandos son ejecutados como una sola operación aislada.
- El comando EXEC ejecuta todos los comandos en la transacción, por lo que si un cliente pierde la conexión al servidor en el contexto de una transacción antes de llamar al comando EXEC, ninguna de las operaciones serán realizadas. En cambio, si el comando EXEC se llega a ejecutar, todas las operaciones son realizadas.

7. ¿Redis tiene persistencia?

Redis cuenta con persistencia, ofreciendo distintas opciones de persistencia:

- RDB (Redis Database): la persistencia de una Base de Datos Redis realiza capturas de los datos en intervalos especificados en el tiempo.
- AOF (Append Only File): la persistencia a través de AOF loguea cada operación de escritura recibida por el servidor. Estas operaciones pueden entonces ser re-ejecutadas de nuevo cuando se inicia el servidor, reconstruyendo los datos originales. Los comandos son logueados usando el mismo formato que el protocolo de Redis.
- RDB + AOF: se pueden combinar ambas técnicas en la misma instancia.

Cabe aclarar, que Redis permite la opción de “No Persistencia”, donde se deshabilita la persistencia completamente. Generalmente se utiliza esta opción cuando se utiliza Redis como una caché.

8. ¿Cuáles son los principales usos de Redis?

- **Búsqueda de texto completo (Full-Text Search)**

- Redis permite indexar texto para hacer búsquedas rápidas con palabras clave, frases, coincidencias parciales, y búsquedas avanzadas (booleanas, de proximidad).
- Ideal para motores de búsqueda internos en apps o sitios web.
- Filtrado y búsqueda por atributos (Faceted Search)
 - Se puede filtrar resultados no solo por texto, sino también por atributos estructurados como categorías, rangos de fechas, etiquetas, etc.
 - Útil para tiendas online, catálogos, inventarios con múltiples filtros.
- Búsqueda geoespacial
 - Redis soporta índices geo-espaciales que permiten consultas rápidas de proximidad, dentro de radios, y cálculos de distancias.
 - Aplicaciones como localización de tiendas, servicios cercanos o seguimiento de vehículos.
- Búsqueda en tiempo real
 - Redis puede procesar grandes volúmenes de datos con baja latencia, ideal para escenarios donde los datos y resultados deben actualizarse en tiempo real.
 - Ejemplos: sistemas de recomendación, análisis en vivo, monitorización.
- Consultas agregadas y analíticas
 - Soporte para agregaciones básicas (conteos, sumas, promedios) sobre datos indexados.
 - Útil para dashboards y reportes rápidos sin necesidad de bases de datos analíticas complejas.
- Integración con modelos de datos complejos
 - Redis Search permite consultas en datos estructurados, combinando texto, números, fechas, booleanos.
 - Se puede combinar con otros módulos para extender funcionalidades, por ejemplo RedisJSON.
- Complemento para bases de datos tradicionales
 - Redis actúa como una capa rápida de búsqueda y consulta encima de bases de datos relacionales o NoSQL, mejorando tiempos de respuesta.

Parte 2: Manejo Simple de Valores String

1. Agregue una clave package con el valor “Bariloche 3 days”

Para realizar esto, introduzco el comando SET con los valores dados. Los comandos ejecutados fueron:

- SET package “Bariloche 3 days”

```
127.0.0.1:6379[1]> SET package "Bariloche 3 days"  
OK  
127.0.0.1:6379[1]>
```

2. Agregue una clave user con el valor “Turismo BD2”. Obtenga el valor de la clave user

Para realizar esto, introduzco el comando SET con los valores dados, y luego hago un GET de la clave ingresada. Los comandos ejecutados fueron:

- SET user “Turismo BD2”
- GET user

```
127.0.0.1:6379[1]> SET user "Turismo BD2"  
OK  
127.0.0.1:6379[1]> GET user  
"Turismo BD2"
```

3. Obtenga todos los valores de claves almacenadas

Para realizar esto, obtengo todas la claves mediante “KEYS *” y luego hago un GET de cada clave. Los comandos ejecutados fueron:

- KEYS *
- GET user
- GET package

```
127.0.0.1:6379[1]> KEYS *  
1) "user"  
2) "package"  
127.0.0.1:6379[1]> GET user  
"Turismo BD2"  
127.0.0.1:6379[1]> GET package  
"Bariloche 3 days"
```

4. Agregue una clave user con el valor “Cronos Turismo” ¿Cuál es el valor actual de la clave user?

Para realizar esto, hago un SET con los datos dados, y luego hago un GET de user. Los comandos ejecutados fueron:

- SET user “Cronos Turismo”
- GET user

El valor actual de la clave “user” es el último ingresado, es decir, “Cronos Turismo”. De esta forma, el valor apuntado por la clave anterior fue sobrescrito, y lógicamente, no hay dos claves “user”.

```
127.0.0.1:6379[1]> SET user "Cronos Turismo"
OK
127.0.0.1:6379[1]> GET user
"Cronos Turismo"
127.0.0.1:6379[1]> KEYS *
1) "user"
2) "package"
```

5. Concatene “ S.A.” a la clave user. ¿Cuál es el valor actual de la clave user?

Para realizar esto, introduzco el comando APPEND con la clave del valor que se le quiere agregar datos, y el dato a agregar. Los comandos ejecutados fueron:

- APPEND user “ S.A”
- GET user

El valor actual luego de ejecutar los comandos dados, fue “Cronos Turismo S.A”. Cabe aclarar, que luego de ejecutar “APPEND user “ S.A””, se obtuvo la salida “(integer) 18”. Esto quiere decir que la longitud de la cadena almacenada es de 18 caracteres.

```
127.0.0.1:6379[1]> APPEND user " S.A"
(integer) 18
127.0.0.1:6379[1]> GET user
"Cronos Turismo S.A"
```

6. Elimine la clave user.

Para realizar esto, introduzco el comando DEL con la clave user. Los comandos ejecutados fueron:

- DEL user
- KEYS *

```
127.0.0.1:6379[1]> DEL user  
(integer) 1  
127.0.0.1:6379[1]> KEYS *  
1) "package"
```

7. ¿Qué valor retorna si queremos obtener la clave user?

Para realizar esto, introduzco el comando GET con la clave user. Los comandos ejecutados fueron:

- GET user
- KEYS *

```
127.0.0.1:6379[1]> GET user  
(nil)  
127.0.0.1:6379[1]> KEYS *  
1) "package"
```

El valor que retorno fue “(nil)”, que es null en otras palabras, es decir, que no devolvió nada, un objeto vacío.

Parte 3: Manejo Simple de Valores Numéricos

1. Verificar si existe la clave visits

Para realizar esto, introduzco el comando EXISTS con la clave dada, el cual devolverá “(integer) 1” si es que existe, o “(integer) 0” en caso de que no exista. El comando ejecutado fue:

- EXISTS visits

```
127.0.0.1:6379[1]> EXISTS visits  
(integer) 0
```

2. Agregue una clave visits con el valor 0

Para realizar esto, introduzco el comando SET con los valores dados. Los comandos ejecutados fueron:

- SET visits 0
- EXISTS visits
- GET visits

```
127.0.0.1:6379[1]> SET visits 0
OK
127.0.0.1:6379[1]> EXISTS visits
(integer) 1
127.0.0.1:6379[1]> GET visits
"0"
```

3. Incremente en 1 visits ¿Cuál es el valor actual de la clave visits?

Para realizar esto, introduzco el comando INCR, el cual toma una clave e incrementa su valor en una unidad en caso que sea un string que contenga solo números. Los comandos ejecutados fueron:

- GET visits
- INCR visits
- GET visits

```
127.0.0.1:6379[1]> GET visits
"0"
127.0.0.1:6379[1]> INCR visits
(integer) 1
127.0.0.1:6379[1]> GET visits
"1"
```

De esta forma, el valor actual de la clave visits es 1.

4. Incremente en 5 visits. ¿Cuál es el valor actual de la clave visits?

Para realizar esto, introduzco el comando INCRBY, el cual toma una clave y el valor que se desea incrementar al valor de dicha clave en caso que sea un string que contenga solo números. Los comandos ejecutados fueron:

- GET visits
- INCRBY visits 5
- GET visits

```
127.0.0.1:6379[1]> GET visits
"1"
127.0.0.1:6379[1]> INCRBY visits 5
(integer) 6
127.0.0.1:6379[1]> GET visits
"6"
```

De esta forma, el valor actual de la clave visits es 6.

5. Decremento en 1 visits ¿Cuál es el valor actual de la clave visits?

Para realizar esto, introduzco el comando DECR, el cual toma una clave y decrementa su valor en una unidad en caso que sea un string que contenga solo números. Los comandos ejecutados fueron:

- GET visits
- DECR visits
- GET visits

```
127.0.0.1:6379[1]> GET visits
"6"
127.0.0.1:6379[1]> DECR visits
(integer) 5
127.0.0.1:6379[1]> GET visits
"5"
```

De esta forma, el valor actual de la clave visits es 5.

6. Incremente en 2 visits. ¿Cuál es el valor actual de la clave visits?

Para realizar esto, introduzco el comando INCRBY utilizado anteriormente. Los comandos ejecutados fueron:

- GET visits
- INCRBY visits 2
- GET visits

```
127.0.0.1:6379[1]> GET visits
"5"
127.0.0.1:6379[1]> INCRBY visits 2
(integer) 7
127.0.0.1:6379[1]> GET visits
"7"
```

De esta forma, el valor actual de la clave visits es 7.

7. Agregue una clave “value package” con el valor 539789.32

Para este caso, introduzco el comando SET con los valores dados. Los comandos ejecutados fueron:

- EXISTS “value package”
- SET “value package” 539789.32
- GET “value package”

```
127.0.0.1:6379[1]> EXISTS "value package"
(integer) 0
127.0.0.1:6379[1]> SET "value package" 539789.32
OK
127.0.0.1:6379[1]> GET "value package"
"539789.32"
```

8. Incremente en 20000 la clave “value package”. ¿Cuál es el valor actual de “value package”?

Para realizar esto, introduzco el comando INCRBYFLOAT, el cual funciona igual que INCRBY, solo que para números que contengan decimales, ya que el string no solo contendrá números. Los comandos ejecutados fueron:

- GET “value package”
- INCRBYFLOAT “value package” 20000
- GET “value package”

```
127.0.0.1:6379[1]> GET "value package"
"539789.32"
127.0.0.1:6379[1]> INCRBYFLOAT "value package" 20000
"559789.3199999999999999318"
127.0.0.1:6379[1]> GET "value package"
"559789.3199999999999999318"
```

De esta forma, el valor actual de “value package” es 559789.3199999999999999318.

9. ¿Cuál es el tipo de datos de “value package”, visits y user?

Para realizar esto, introduzco el comando TYPE, el cual devuelve los tipos de datos que contienen las claves que se le pasan si estas existen. Los comandos ejecutados fueron:

- KEYS *
- TYPE visits
- TYPE “value package”
- TYPE user

```

127.0.0.1:6379[1]> KEYS *
1) "visits"
2) "package"
3) "value package"
127.0.0.1:6379[1]> TYPE visits
string
127.0.0.1:6379[1]> TYPE "value package"
string
127.0.0.1:6379[1]> TYPE user
none

```

De esta forma, el valor de las claves fueron string, ya que como mencionamos en la Parte 1, los datos de texto plano se almacenan como string, realizandose los chequeos necesarios ante una operación específica para un tipo de dato en particular. Esto se pudo ver cuando se incrementaban valores para los enteros y reales, y se decrementaban valores enteros. Cabe aclarar, que para los reales no existe el comando para decrementar, pero introduciendo un valor negativo con el INCRBYFLOAT, hará que se reste el valor ingresado.

Además, el comando “TYPE user” devolvió “none” ya que la clave fue borrada en la Parte 2, por lo que devolvió este tipo de dato que significa que no existe un valor mediante esa clave.

Parte 4: Manejo de Claves

1. Obtenga todas las claves que empiecen con v

Para realizar esto, introduzco el comando KEYS, con el valor dado como inicial y seguido de * para que pueda encontrar todas las variaciones. Los comandos ejecutados fueron:

- KEYS *
- KEYS v*

```

127.0.0.1:6379[1]> KEYS *
1) "visits"
2) "package"
3) "value package"
127.0.0.1:6379[1]> KEYS v*
1) "visits"
2) "value package"

```

2. Obtenga todas las claves que contengan la “t”

Para realizar esto, introduzco el comando KEYS, con el valor dado entre *’s para que pueda encontrar todas las variaciones. Los comandos ejecutados fueron:

- KEYS *
- KEYS *t*

```
127.0.0.1:6379[1]> KEYS *
1) "visits"
2) "package"
3) "value package"
127.0.0.1:6379[1]> KEYS *t*
1) "visits"
```

3. Obtenga todas las claves que terminen con “age”

Para realizar esto, introduzco el comando KEYS, con el valor dado luego de una *, para que pueda encontrar todas las variaciones. Los comandos ejecutados fueron:

- KEYS *
- KEYS *age

```
127.0.0.1:6379[1]> KEYS *
1) "visits"
2) "package"
3) "value package"
127.0.0.1:6379[1]> KEYS *age
1) "package"
2) "value package"
```

4. Renombre la clave “package” por “bariloché package”

Para realizar esto, introduzco el comando RENAME, seguido de la clave que se quiere cambiar y su nuevo nombre. Los comandos ejecutados fueron:

- KEYS package
- RENAME package “bariloché package”
- KEYS *

```
127.0.0.1:6379[1]> KEYS package
1) "package"
127.0.0.1:6379[1]> RENAME package "bariloché package"
OK
127.0.0.1:6379[1]> KEYS *
1) "visits"
2) "bariloché package"
3) "value package"
```

5. Si quisiera evitar renombrar una clave por otra existente, ¿qué comando utilizaría?

Para este caso, utilizaría RENAMENX, el cual toma como argumentos una clave existente y el nuevo nombre que se le asignará, y en el orden dado. Sin embargo, el

nuevo nombre será asignada a la clave pasada en caso de que ya no exista una clave con ese nombre ingresado.

```
127.0.0.1:6379[1]> KEYS *
1) "visits"
2) "bariloché package"
3) "value package"
127.0.0.1:6379[1]> RENAMENX visits "value package"
(integer) 0
127.0.0.1:6379[1]> KEYS *
1) "visits"
2) "bariloché package"
3) "value package"
127.0.0.1:6379[1]> RENAMENX visits visits_new
(integer) 1
127.0.0.1:6379[1]> KEYS *
1) "bariloché package"
2) "visits_new"
3) "value package"
```

Como se puede ver, en este caso intenté renombrar “visits” por “value package”, donde este último era una clave existente, por lo que devolvió un “(integer) 0” indicando que la acción no se pudo concretar. Pero al querer renombrar “visits” por “visits_new”, el cual era una clave que no existía, devolvió un “(integer) 1” indicando que la acción se pudo concretar.

6. Elimine todas las claves

Para realizar esto, introduzco el comando FLUSHDB, el cual borra todas las claves de la Base de Datos actual, es decir, desde la que se ejecuta el comando. Así, los comandos ejecutados fueron:

- KEYS *
- FLUSHDB
- KEYS *

```
127.0.0.1:6379[1]> KEYS *
1) "bariloché package"
2) "visits_new"
3) "value package"
127.0.0.1:6379[1]> FLUSHDB
OK
127.0.0.1:6379[1]> KEYS *
(empty array)
```